ಮಾದರಿ ಕಾರ್ಯಕ್ಷಮತೆ ಅಳೆಯುವುದು ಮತ್ತು Keras ನಲ್ಲಿ ಮೊದಲ ನ್ಯೂರಲ್ ನೆಟ್‌ವರ್ಕ್

Confusion Matrix, Precision, Recall, F1, ROC, Overfitting ಮತ್ತು MNIST ಅಂಕೆ ಗುರುತಿಸುವಿಕೆ

ಸರಳ ಕನ್ನಡದಲ್ಲಿ, ದಿನನಿತ್ಯದ ಉದಾಹರಣೆಗಳು ಮತ್ತು ಹಂತ-ಹಂತದ ಲೆಕ್ಕಗಳೊಂದಿಗೆ

“Lecture 5: Classification Performance Metrics”
ಉಪನ್ಯಾಸ ಸ್ಲೈಡ್‌ಗಳನ್ನು ಆಧರಿಸಿದ ಕನ್ನಡ ಅಧ್ಯಯನ ಮಾರ್ಗದರ್ಶಿ

ಪರಿವಿಡಿ

೧. ದೊಡ್ಡ ಚಿತ್ರ: ಈ ಉಪನ್ಯಾಸದ ನಕ್ಷೆ

೨. Confusion Matrix — ಕೋವಿಡ್ ಪರೀಕ್ಷೆಯ ಕಥೆ

೩. ಲೆಕ್ಕ ಹಾಕೋಣ: 1000 ಜನರ ಪರೀಕ್ಷೆ

೪. Accuracy ಯ ಮಿತಿ — 99.9% ಇದ್ದರೂ ಮಾದರಿ ನಿರುಪಯುಕ್ತ!

೫. Precision ಮತ್ತು Recall, Threshold ಸಮತೋಲನ

೬. F1 Score — ಎರಡರ ಸಾಮರಸ್ಯ

೭. ROC ಮತ್ತು AUC

೮. ಬಹು-ವರ್ಗ Confusion Matrix

೯. Train / Validation / Test ಮತ್ತು Hyperparameters

೧೦. Learning Curves, Overfitting ಮತ್ತು Early Stopping

೧೧. Keras: Sequential, Input, Flatten, Dense

೧೨. ಮಾದರಿ ನಿರ್ಮಾಣದ ಜೀವನಚಕ್ರ ಮತ್ತು MNIST ಸಮಸ್ಯೆ

೧೩. ಕೋಡ್ ಹಂತ-ಹಂತವಾಗಿ: ಡೇಟಾದಿಂದ ಭವಿಷ್ಯವಾಣಿವರೆಗೆ

೧೪. ತರಬೇತಿ ಫಲಿತಾಂಶಗಳ ವಿಶ್ಲೇಷಣೆ (ಸ್ಲೈಡ್‌ನ ನಿಜವಾದ ಸಂಖ್ಯೆಗಳು)

೧೫. ಸಾರಾಂಶ, ಪದಕೋಶ ಮತ್ತು ಅಭ್ಯಾಸ ಪ್ರಶ್ನೆಗಳು

೧. ದೊಡ್ಡ ಚಿತ್ರ: ಈ ಉಪನ್ಯಾಸದ ನಕ್ಷೆ

ಮಾದರಿಯನ್ನು ತರಬೇತಿ ಮಾಡಿದ ನಂತರ ಮೂರು ಪ್ರಶ್ನೆಗಳು ಏಳುತ್ತವೆ: **(೧) ಇದು ಎಷ್ಟು ಒಳ್ಳೆಯದು? (೨) ಹೊಸ ದತ್ತಾಂಶದಲ್ಲೂ ಒಳ್ಳೆಯದಾಗಿ ಕೆಲಸ ಮಾಡುತ್ತದೆಯೇ? (೩) ಇದನ್ನು ಕೋಡ್‌ನಲ್ಲಿ ಹೇಗೆ ನಿರ್ಮಿಸುವುದು?** ಈ ಉಪನ್ಯಾಸ ಈ ಮೂರಕ್ಕೂ ಉತ್ತರಿಸುತ್ತದೆ.

ಭಾಗ	ಪ್ರಶ್ನೆ	ಮುಖ್ಯ ವಿಷಯಗಳು
೧. ಕಾರ್ಯಕ್ಷಮತೆ ಮೆಟ್ರಿಕ್ಸ್	ಮಾದರಿ ಎಷ್ಟು ಸರಿ?	Confusion matrix, Accuracy, Precision, Recall, F1, ROC/AUC
೨. ಕಲಿಕೆಯ ವಕ್ರರೇಖೆಗಳು	ಕಲಿಕೆ ಆರೋಗ್ಯಕರವೇ?	Train/validation/test, learning curves, early stopping
೩. Overfitting / Underfitting	ಉರು ಹೊಡೆದಿದೆಯೇ?	Parameters vs hyperparameters
೪. Keras ವಿನ್ಯಾಸ	ನೆಟ್‌ವರ್ಕ್ ಹೇಗೆ ಕಟ್ಟುವುದು?	Sequential, Input, Flatten, Dense
೫. MNIST ಪ್ರಾಜೆಕ್ಟ್	ಕಂಪ್ಯೂಟರ್‌ಗೆ ಅಂಕೆ ಓದಲು ಕಲಿಸುವುದು	28×28 ಚಿತ್ರ → 784 ಇನ್‌ಪುಟ್ → 10 ಔಟ್‌ಪುಟ್

೨. Confusion Matrix — ಕೋವಿಡ್ ಪರೀಕ್ಷೆಯ ಕಥೆ

Confusion matrix (ಗೊಂದಲ ಮ್ಯಾಟ್ರಿಕ್ಸ್) ಎಂದರೆ ವರ್ಗೀಕರಣ ಮಾದರಿಯ ಊಹೆಗಳನ್ನು ನಿಜವಾದ ಉತ್ತರಗಳೊಂದಿಗೆ ಹೋಲಿಸಿ ತೋರಿಸುವ ಕೋಷ್ಟಕ. ಮಾದರಿ ಎಲ್ಲಿ ಸರಿ, ಎಲ್ಲಿ ತಪ್ಪು, ಯಾವ ಬಗೆಯ ತಪ್ಪು ಮಾಡುತ್ತಿದೆ ಎಂದು ಒಂದೇ ನೋಟದಲ್ಲಿ ತಿಳಿಯುತ್ತದೆ.

> **ಸನ್ನಿವೇಶ: ಕೋವಿಡ್ ಪರೀಕ್ಷೆ**
> ರೋಗಿ (ಕೋವಿಡ್ ಇದೆ) = **1 (Positive)**, ಆರೋಗ್ಯವಂತ (ಕೋವಿಡ್ ಇಲ್ಲ) = **0 (Negative)**. ಪರೀಕ್ಷೆ (ಅಥವಾ AI ಮಾದರಿ) ಪ್ರತಿ ವ್ಯಕ್ತಿಗೆ "ಪಾಸಿಟಿವ್" ಅಥವಾ "ನೆಗೆಟಿವ್" ಫಲಿತಾಂಶ ಕೊಡುತ್ತದೆ. ಆದರೆ ಪರೀಕ್ಷೆಯೂ ತಪ್ಪು ಮಾಡಬಹುದು. ನಾಲ್ಕು ಸಾಧ್ಯತೆಗಳಿವೆ.

	ಪರೀಕ್ಷೆ ಹೇಳಿದ್ದು: Positive	**ಪರೀಕ್ಷೆ ಹೇಳಿದ್ದು: Negative**
ನಿಜ: ರೋಗಿ (Positive)	**TP** (True Positive) ✓ ರೋಗಿಯನ್ನು ಸರಿಯಾಗಿ "ರೋಗಿ" ಎಂದಿತು	**FN** (False Negative) ✗ ರೋಗಿಯನ್ನು "ಆರೋಗ್ಯವಂತ" ಎಂದಿತು — **ರೋಗ ತಪ್ಪಿಸಿಕೊಂಡಿತು!**
ನಿಜ: ಆರೋಗ್ಯವಂತ (Negative)	**FP** (False Positive) ✗ ಆರೋಗ್ಯವಂತನನ್ನು "ರೋಗಿ" ಎಂದಿತು — ಸುಳ್ಳು ಎಚ್ಚರಿಕೆ	**TN** (True Negative) ✓ ಆರೋಗ್ಯವಂತನನ್ನು ಸರಿಯಾಗಿ "ಆರೋಗ್ಯವಂತ" ಎಂದಿತು

> **ಹೆಸರುಗಳನ್ನು ನೆನಪಿಡುವ ಸೂತ್ರ**
> ಎರಡನೆಯ ಪದ = **ಮಾದರಿ ಏನು ಹೇಳಿತು** (Positive/Negative). ಮೊದಲನೆಯ ಪದ = **ಅದು ಸರಿಯೇ?** (True = ಸರಿ, False = ತಪ್ಪು).
> ಉದಾ: "False Negative" = ಮಾದರಿ "Negative" ಎಂದಿತು, ಆದರೆ ಅದು ತಪ್ಪು (ನಿಜವಾಗಿ ರೋಗಿ ಇದ್ದ).

ಎಚ್ಚರ: ಎರಡು ಬಗೆಯ ಕೋಷ್ಟಕ ವಿನ್ಯಾಸ

ಸ್ಲೈಡ್‌ನಲ್ಲಿ ತೋರಿಸಿದಂತೆ ಇಲ್ಲಿ **ಸಾಲುಗಳು = ನಿಜವಾದ ವರ್ಗ, ಕಾಲಮ್‌ಗಳು = ಊಹೆ** (ಹೆಚ್ಚಿನ ML ಲೈಬ್ರರಿಗಳು ಬಳಸುವ ಕ್ಲಾಸಿಕ್ ರೂಪ). ಕೆಲವು ಪುಸ್ತಕಗಳಲ್ಲಿ ಇದರ ಉಲ್ಟಾ (ಸಾಲು = ಊಹೆ, ಕಾಲಮ್ = ನಿಜ) ಇರುತ್ತದೆ — ಆಗ FN ಮತ್ತು FP ಸ್ಥಾನ ಅದಲು-ಬದಲು ಆಗುತ್ತದೆ. ಯಾವಾಗಲೂ ಯಾವ ವಿನ್ಯಾಸ ಎಂದು ಪರಿಶೀಲಿಸಿ.

೩. ಲೆಕ್ಕ ಹಾಕೋಣ: 1000 ಜನರ ಪರೀಕ್ಷೆ

ಸನ್ನಿವೇಶ (ನಮ್ಮದೇ ಉದಾಹರಣೆ)

ಒಂದು ಶಿಬಿರದಲ್ಲಿ 1000 ಜನರನ್ನು ಪರೀಕ್ಷಿಸಿದರು: ನಿಜವಾಗಿ 100 ಜನ ರೋಗಿಗಳು, 900 ಆರೋಗ್ಯವಂತರು. ಪರೀಕ್ಷೆಯ ಫಲಿತಾಂಶ:

	Predicted Positive	Predicted Negative	ಒಟ್ಟು
ನಿಜ ರೋಗಿ	TP = 80	FN = 20	100
ನಿಜ ಆರೋಗ್ಯವಂತ	FP = 45	TN = 855	900

```
Accuracy    = (TP + TN) / ಒಟ್ಟು    = (80 + 855) / 1000 = 0.935  → 93.5%
Precision   = TP / (TP + FP)          = 80 / (80 + 45)    = 0.64   → 64%
Recall      = TP / (TP + FN)          = 80 / (80 + 20)    = 0.80   → 80%
FPR         = FP / (FP + TN)          = 45 / (45 + 855)   = 0.05   → 5%
Specificity = 1 − FPR                 = 0.95                       → 95%
F1          = 2·P·R / (P + R)         = 2·0.64·0.80 / 1.44 = 0.711
```

ಅರ್ಥ: ನಿಜ ರೋಗಿಗಳಲ್ಲಿ 80% ರನ್ನು ಹಿಡಿಯಿತು (Recall). ಆದರೆ “ಪಾಸಿಟಿವ್” ಎಂದ 125 ಜನರಲ್ಲಿ ಕೇವಲ 64% ನಿಜವಾಗಿ ರೋಗಿಗಳು (Precision) — 45 ಜನ ಸುಮ್ಮನೆ ಭಯಪಟ್ಟರು.

೪. Accuracy ಯ ಮಿತಿ — 99.9% ಇದ್ದರೂ ಮಾದರಿ ನಿರುಪಯುಕ್ತ!

Accuracy = ಎಲ್ಲ ಊಹೆಗಳಲ್ಲಿ ಸರಿಯಾದವುಗಳ ಅನುಪಾತ: `(TP + TN) / (TP + TN + FP + FN)`.

ಸ್ಲೈಡ್‌ನ ಉದಾಹರಣೆ: ಅಪರೂಪದ ರೋಗ

ಸುಮಾರು 10,000 ಜನರಲ್ಲಿ (ಸ್ಲೈಡ್‌ನ ಲೆಕ್ಕದಲ್ಲಿ ಒಟ್ಟು 9,999) ಕೇವಲ 9 ಜನ ರೋಗಿಗಳು, 9,990 ಜನ ಆರೋಗ್ಯವಂತರು. ಮಾದರಿ ಏನೂ ಯೋಚಿಸದೆ **ಎಲ್ಲರಿಗೂ "ಆರೋಗ್ಯವಂತ" ಎಂದೇ** ಹೇಳುತ್ತದೆ:

	Predicted Positive	Predicted Negative
ನಿಜ ರೋಗಿ	TP = 0	FN = 9
ನಿಜ ಆರೋಗ್ಯವಂತ	FP = 0	TN = 9990

```
Accuracy = (0 + 9990) / (0 + 9990 + 9 + 0) = 9990 / 9999 = 0.99910  → 99.91% !!
Recall   = 0 / (0 + 9)                                              = 0%      (ಒಬ್ಬ ರೋಗಿಯನ್ನೂ ಹಿಡಿಯಲಿಲ್ಲ)
```

ಹೋಲಿಕೆ: ನಿದ್ರಿಸುವ ಕಾವಲುಗಾರ

ಊರಿನಲ್ಲಿ 10,000 ದಿನಗಳಲ್ಲಿ ಕೇವಲ 9 ದಿನ ಕಳ್ಳತನ ಆಗುತ್ತದೆ. ಕಾವಲುಗಾರ ಪ್ರತಿ ದಿನ ಮಲಗಿ "ಇಂದು ಕಳ್ಳತನ ಇಲ್ಲ" ಎಂದು ವರದಿ ಕೊಟ್ಟರೂ 99.9% ಸರಿ! ಆದರೆ ಒಂದೂ ಕಳ್ಳತನ ತಡೆಯಲಿಲ್ಲ. ದತ್ತಾಂಶ ಅಸಮತೋಲನವಾಗಿದ್ದಾಗ (ಒಂದು ವರ್ಗ ಅತಿ ಅಪರೂಪ) Accuracy ಮೋಸ ಮಾಡುತ್ತದೆ. ಆಗ Precision, Recall, F1 ನೋಡಬೇಕು.

೫. Precision ಮತ್ತು Recall, Threshold ಸಮತೋಲನ

ಅಳತೆ	ಸೂತ್ರ	ಕೇಳುವ ಪ್ರಶ್ನೆ
Precision (ನಿಖರತೆ)	TP / (TP + FP)	ಮಾದರಿ "ಪಾಸಿಟಿವ್" ಎಂದವರಲ್ಲಿ ಎಷ್ಟು ಜನ ನಿಜವಾಗಿ ಪಾಸಿಟಿವ್?
Recall / TPR / Sensitivity (ಸ್ಮರಣೆ)	TP / (TP + FN)	ನಿಜವಾಗಿ ಪಾಸಿಟಿವ್ ಇರುವವರಲ್ಲಿ ಎಷ್ಟು ಜನರನ್ನು ಮಾದರಿ ಹಿಡಿಯಿತು?
FPR (ಸುಳ್ಳು ಎಚ್ಚರಿಕೆ ದರ)	FP / (FP + TN)	ನಿಜವಾಗಿ ನೆಗೆಟಿವ್ ಇರುವವರಲ್ಲಿ ಎಷ್ಟು ಜನರನ್ನು ತಪ್ಪಾಗಿ ಪಾಸಿಟಿವ್ ಎಂದಿತು?

ಹೋಲಿಕೆ: ಕೆರೆಗೆ ಬಲೆ ಬೀಸುವುದು

ಕೆರೆಯಲ್ಲಿ 100 ಮೀನುಗಳಿವೆ. ನೀವು ಬಲೆ ಬೀಸಿದಾಗ 125 ವಸ್ತುಗಳು ಸಿಕ್ಕವು — 80 ಮೀನು, 45 ಕಸ-ಕಡ್ಡಿ. **Recall** = ಕೆರೆಯ ಎಲ್ಲ ಮೀನುಗಳಲ್ಲಿ ಎಷ್ಟನ್ನು ಹಿಡಿದೆ = 80/100. **Precision** = ಬಲೆಯಲ್ಲಿ ಬಿದ್ದವುಗಳಲ್ಲಿ ಎಷ್ಟು ನಿಜವಾದ ಮೀನು = 80/125. ದೊಡ್ಡ ಬಲೆ ಹಾಕಿದರೆ ಹೆಚ್ಚು ಮೀನು (Recall ↑) ಆದರೆ ಹೆಚ್ಚು ಕಸ (Precision ↓). ಚಿಕ್ಕ, ಆಯ್ದ ಬಲೆ ಹಾಕಿದರೆ ಶುದ್ಧ ಮೀನು (Precision ↑) ಆದರೆ ಕೆಲವು ಮೀನು ತಪ್ಪಿಹೋಗುತ್ತವೆ (Recall ↓).

ಯಾವುದು ಹೆಚ್ಚು ಮುಖ್ಯ?

ಸನ್ನಿವೇಶ	ದುಬಾರಿ ತಪ್ಪು	ಹೆಚ್ಚು ಗಮನಿಸಬೇಕಾದದ್ದು
ಕ್ಯಾನ್ಸರ್ / ಕೋವಿಡ್ ಸ್ಕ್ರೀನಿಂಗ್	FN — ರೋಗ ತಪ್ಪಿಸಿಕೊಳ್ಳುವುದು ಪ್ರಾಣಕ್ಕೆ ಅಪಾಯ	**Recall**
ಸ್ಪ್ಯಾಮ್ ಫಿಲ್ಟರ್	FP — ಮುಖ್ಯ ಇಮೇಲ್ ಅನ್ನು ಸ್ಪ್ಯಾಮ್ ಎಂದು ಅಳಿಸುವುದು	**Precision**
ಬ್ಯಾಂಕ್ ವಂಚನೆ ಪತ್ತೆ	FN — ವಂಚನೆ ತಪ್ಪಿಸಿಕೊಳ್ಳುವುದು (ಆದರೆ ಅತಿ FP ಗ್ರಾಹಕರನ್ನು ಕೆರಳಿಸುತ್ತದೆ)	ಎರಡರ ಸಮತೋಲನ (F1)

Threshold (ಮಿತಿ) ಬದಲಿಸಿದರೆ ಏನಾಗುತ್ತದೆ? — ಸ್ಲೈಡ್‌ನ “5” ಅಂಕೆ ಉದಾಹರಣೆ

ಮಾದರಿ ಪ್ರತಿ ಚಿತ್ರಕ್ಕೆ “ಇದು 5 ಆಗಿರುವ ಸಾಧ್ಯತೆ” ಎಂಬ ಸ್ಕೋರ್ ಕೊಡುತ್ತದೆ; ನಾವು ಒಂದು ಮಿತಿ ಇಟ್ಟು “ಈ ಸ್ಕೋರ್‌ಗಿಂತ ಹೆಚ್ಚಿದ್ದರೆ ‘5’” ಎನ್ನುತ್ತೇವೆ. ಹತ್ತು ಚಿತ್ರಗಳನ್ನು ಸ್ಕೋರ್ ಕ್ರಮದಲ್ಲಿ ಜೋಡಿಸಿದ್ದೇವೆ; ಇವುಗಳಲ್ಲಿ ನಿಜವಾದ “5” ಗಳು 6.



ಹಸಿರು 5 = ನಿಜವಾದ “5” (ಒಟ್ಟು 6), ಕೆಂಪು × = ಬೇರೆ ಅಂಕೆ (ಒಟ್ಟು 4)

ಚಿತ್ರ ೧: ಮಿತಿಯನ್ನು ಬಲಕ್ಕೆ ಸರಿಸಿದಂತೆ ಕಡಿಮೆ ಚಿತ್ರಗಳು “5” ಎಂದು ಗುರುತಿಸಲ್ಪಡುತ್ತವೆ

ಮಿತಿ	“5” ಎಂದ ಚಿತ್ರಗಳು	ಅದರಲ್ಲಿ ನಿಜ 5	Precision	Recall
A (ಸಡಿಲ)	8	6	6/8 = **75%**	6/6 = **100%**
B (ಮಧ್ಯಮ)	5	4	4/5 = **80%**	4/6 = **67%**
C (ಕಠಿಣ)	3	3	3/3 = **100%**	3/6 = **50%**

> **ಪಾಠ**
>
> ಮಿತಿ ಕಠಿಣವಾದಂತೆ Precision ಏರುತ್ತದೆ (ಹೆಚ್ಚು ಖಚಿತವಾದವುಗಳನ್ನು ಮಾತ್ರ ಹೇಳುತ್ತೇವೆ) ಆದರೆ Recall ಇಳಿಯುತ್ತದೆ (ಕೆಲವು ನಿಜ 5 ಗಳು ತಪ್ಪಿ ಹೋಗುತ್ತವೆ). ಎರಡೂ ಒಟ್ಟಿಗೆ 100% ಆಗುವುದು ಬಹಳ ಅಪರೂಪ — ಇದೇ **precision–recall trade-off**. ಮಿತಿಯನ್ನು ನಿಮ್ಮ ಅಗತ್ಯಕ್ಕೆ ತಕ್ಕಂತೆ ಆರಿಸಬೇಕು.

೬. F1 Score — ಎರಡರ ಸಾಮರಸ್ಯ

Precision ಮತ್ತು Recall ಎರಡನ್ನೂ ಒಂದೇ ಸಂಖ್ಯೆಯಲ್ಲಿ ಸಾರಾಂಶ ಮಾಡಲು **F1 score** ಬಳಸುತ್ತೇವೆ — ಇದು ಅವುಗಳ **harmonic mean (ಹಾರ್ಮೋನಿಕ್ ಸರಾಸರಿ)**:

```
F1 = 2 × Precision × Recall / (Precision + Recall) = TP / ( TP + (FP + FN)/2 )
```

ಮಿತಿ	Precision	Recall	F1	ಸಾಮಾನ್ಯ ಸರಾಸರಿ
A	0.75	1.00	**0.857**	0.875
B	0.80	0.667	**0.727**	0.733
C	1.00	0.50	**0.667**	0.750

ಈ ಮೂರು ಮಿತಿಗಳಲ್ಲಿ **A** ಅತ್ಯುತ್ತಮ ಸಮತೋಲನ (F1 = 0.857).

> **ಸಾಮಾನ್ಯ ಸರಾಸರಿ ಬದಲು ಹಾರ್ಮೋನಿಕ್ ಏಕೆ?**
> ಒಂದು ಮಾದರಿ: Precision = 1.0, Recall = 0.1 (ಕೇವಲ 10% ರೋಗಿಗಳನ್ನು ಹಿಡಿಯುತ್ತದೆ, ಆದರೆ ಹಿಡಿದವರೆಲ್ಲ ನಿಜ).
>
> ```
> ಸಾಮಾನ್ಯ ಸರಾಸರಿ = (1.0 + 0.1) / 2          = 0.55   (ಚೆನ್ನಾಗಿದೆ ಎನಿಸುತ್ತದೆ — ಮೋಸ!)
> F1               = 2 × 1.0 × 0.1 / (1.0 + 0.1) = 0.18   (ನಿಜ ಸ್ಥಿತಿ)
> ```
>
> ಹಾರ್ಮೋನಿಕ್ ಸರಾಸರಿ ಚಿಕ್ಕ ಸಂಖ್ಯೆಯ ಕಡೆ ಎಳೆಯುತ್ತದೆ; ಒಂದು ಅಳತೆ ಕೆಟ್ಟಿದ್ದರೆ ಇನ್ನೊಂದು ಚೆನ್ನಾಗಿದ್ದರೂ F1 ಕುಸಿಯುತ್ತದೆ. *ಹೋಲಿಕೆ:* ಒಂದು ಕೈ ಬಲಿಷ್ಠ, ಇನ್ನೊಂದು ದುರ್ಬಲ ಇರುವ ಕುಸ್ತಿಪಟು ಒಟ್ಟಾರೆ ದುರ್ಬಲ.

೭. ROC ಮತ್ತು AUC

ಮೇಲೆ ನೋಡಿದಂತೆ ಒಂದೊಂದು threshold ಗೆ ಒಂದೊಂದು ಫಲಿತಾಂಶ. **ROC (Receiver Operating Characteristic) ವಕ್ರರೇಖೆ** ಎಲ್ಲ ಮಿತಿಗಳಲ್ಲಿ ಮಾದರಿ ಧನಾತ್ಮಕ-ಋಣಾತ್ಮಕಗಳನ್ನು ಎಷ್ಟು ಚೆನ್ನಾಗಿ ಬೇರ್ಪಡಿಸುತ್ತದೆ ಎಂದು ಒಂದೇ ಚಿತ್ರದಲ್ಲಿ ತೋರಿಸುತ್ತದೆ.

- **Y ಅಕ್ಷ: TPR** (Recall / Sensitivity) = TP/(TP+FN)
- **X ಅಕ್ಷ: FPR** = FP/(FP+TN)
- **Specificity** = 1 − FPR (ಆರೋಗ್ಯವಂತರನ್ನು ಸರಿಯಾಗಿ ಗುರುತಿಸುವ ಸಾಮರ್ಥ್ಯ)
- **AUC (Area Under Curve)** = ರೇಖೆಯ ಕೆಳಗಿನ ವಿಸ್ತೀರ್ಣ. 1.0 = ಪರಿಪೂರ್ಣ, 0.5 = ನಾಣ್ಯ ಚಿಮ್ಮಿ ಊಹಿಸಿದಂತೆ (ಕರ್ಣ ರೇಖೆ).

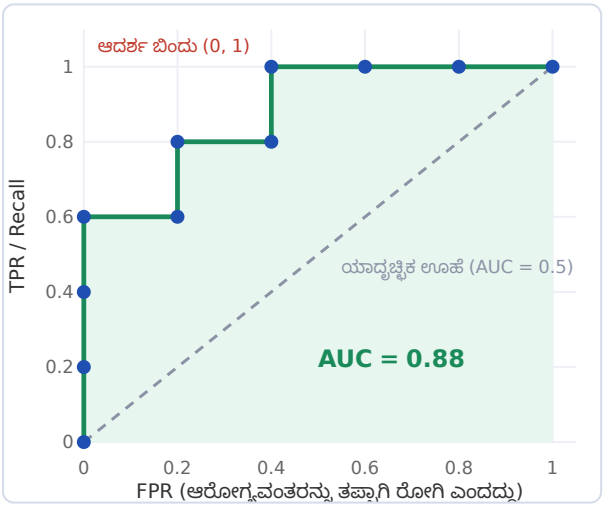
ಹಂತ-ಹಂತದ ಉದಾಹರಣೆ: 10 ರೋಗಿಗಳ ಅಪಾಯ ಅಂಕ

ಮಾದರಿ ಪ್ರತಿ ವ್ಯಕ್ತಿಗೆ 0 ರಿಂದ 1 ರ ಅಪಾಯ ಅಂಕ ಕೊಟ್ಟಿದೆ. 5 ಜನ ನಿಜವಾಗಿ ರೋಗಿಗಳು, 5 ಜನ ಆರೋಗ್ಯವಂತರು:

ನಿಜ ರೋಗಿಗಳ ಅಂಕ	0.95, 0.85, 0.80, 0.65, 0.45
ಆರೋಗ್ಯವಂತರ ಅಂಕ	0.70, 0.55, 0.40, 0.30, 0.10

ಮಿತಿ (ಅಂಕ ≥ ಮಿತಿ ಆದರೆ “ರೋಗಿ”) ಬದಲಿಸುತ್ತಾ ಹೋದಂತೆ:

ಮಿತಿ	TP	FP	TPR = TP/5	FPR = FP/5
0.90	1	0	0.2	0.0
0.75	3	0	0.6	0.0
0.60	4	1	0.8	0.2
0.50	4	2	0.8	0.4
0.35	5	3	1.0	0.6
0.20	5	4	1.0	0.8
0.00	5	5	1.0	1.0



ಚಿತ್ರ ೨: ಈ ಉದಾಹರಣೆಯ ROC ವಕ್ರರೇಖೆ (ಹಸಿರು ಭಾಗ = AUC)

ಪ್ರತಿ ಸಾಲಿನ (FPR, TPR) ಒಂದು ಬಿಂದು. ಬಿಂದುಗಳನ್ನು ಜೋಡಿಸಿದರೆ ROC ವಕ್ರರೇಖೆ. ಆದರ್ಶ ಬಿಂದು ಮೇಲಿನ ಎಡ ಮೂಲೆ (0, 1) — ಎಲ್ಲ ರೋಗಿಗಳನ್ನು ಹಿಡಿದು ಯಾವ ಆರೋಗ್ಯವಂತರನ್ನೂ ತಪ್ಪಾಗಿ ಗುರುತಿಸದಿರುವುದು.

AUC = 0.88. ಅರ್ಥ: ಒಬ್ಬ ಯಾದೃಚ್ಛಿಕ ರೋಗಿ ಮತ್ತು ಒಬ್ಬ ಯಾದೃಚ್ಛಿಕ ಆರೋಗ್ಯವಂತನನ್ನು ಆರಿಸಿದರೆ, ರೋಗಿಗೆ ಹೆಚ್ಚು ಅಂಕ ಸಿಗುವ ಸಾಧ್ಯತೆ 88%. (25 ಜೋಡಿಗಳಲ್ಲಿ 22 ಜೋಡಿಗಳಲ್ಲಿ ರೋಗಿಯ ಅಂಕ ಹೆಚ್ಚು: 22/25 = 0.88.)

> **ROC/AUC ಯ ಅನುಕೂಲ**
>
> ಒಂದೇ ಮಿತಿಯ ಮೇಲೆ ಅವಲಂಬಿತವಲ್ಲ — ಮಾದರಿಯ “ಬೇರ್ಪಡಿಸುವ ಸಾಮರ್ಥ್ಯ”ವನ್ನು ಒಟ್ಟಾರೆ ಅಳೆಯುತ್ತದೆ. ಎರಡು ಮಾದರಿಗಳನ್ನು ಹೋಲಿಸಲು ಉತ್ತಮ. (ಬಹಳ ಅಸಮತೋಲನ ದತ್ತಾಂಶದಲ್ಲಿ Precision–Recall ವಕ್ರರೇಖೆಯನ್ನೂ ನೋಡುವುದು ಒಳ್ಳೆಯದು.)

೮. ಬಹು-ವರ್ಗ Confusion Matrix

ಎರಡಕ್ಕಿಂತ ಹೆಚ್ಚು ವರ್ಗ ಇದ್ದರೆ (ಉದಾ: A, B, C) ಮ್ಯಾಟ್ರಿಕ್ಸ್ ದೊಡ್ಡದಾಗುತ್ತದೆ. **ಸಾಲು = ನಿಜವಾದ ವರ್ಗ, ಕಾಲಮ್ = ಊಹಿಸಿದ ವರ್ಗ.** ಸೆಲ್ [i, j] = ನಿಜ ವರ್ಗ i ಆಗಿದ್ದು, j ಎಂದು ಊಹಿಸಿದ ಉದಾಹರಣೆಗಳ ಸಂಖ್ಯೆ. **ಕರ್ಣ (diagonal) = ಸರಿಯಾದ ಊಹೆಗಳು**; ಕರ್ಣದ ಹೊರಗಿನವು = ತಪ್ಪುಗಳು.

ಸ್ಲೈಡ್‌ನ ಉದಾಹರಣೆ: 3 ವರ್ಗಗಳು (A, B, C)

	Pred A	Pred B	Pred C	ಸಾಲಿನ ಒಟ್ಟು
True A	8	1	0	9
True B	2	6	1	9
True C	0	1	7	8
ಕಾಲಮ್ ಒಟ್ಟು	10	8	8	26

```
Accuracy = (TP1 + TP2 + TP3) / ಒಟ್ಟು = (8 + 6 + 7) / 26 = 21 / 26 = 0.808  → 80.8%
```

ಪ್ರತಿ ವರ್ಗದ ಅಳತೆಗಳು (ಆ ವರ್ಗವನ್ನು "Positive" ಎಂದು ಪರಿಗಣಿಸಿ)

Recall = ಕರ್ಣ / ಸಾಲಿನ ಒಟ್ಟು; Precision = ಕರ್ಣ / ಕಾಲಮ್ ಒಟ್ಟು.

ವರ್ಗ	Recall	Precision	F1
A	8/9 = 0.889	8/10 = 0.800	0.842
B	6/9 = 0.667	6/8 = 0.750	0.706
C	7/8 = 0.875	7/8 = 0.875	0.875
ಸರಾಸರಿ (macro)	0.810	0.808	—

ವರ್ಗ B ಅತ್ಯಂತ ದುರ್ಬಲ (Recall 0.667): ನಿಜ B ಗಳಲ್ಲಿ 2 ಅನ್ನು A ಎಂದು, 1 ಅನ್ನು C ಎಂದು ತಪ್ಪಾಗಿ ಹೇಳಲಾಗಿದೆ. ಇಂತಹ ವಿವರ ಕೇವಲ Accuracy ಯಲ್ಲಿ ಸಿಗುವುದಿಲ್ಲ — ಅದಕ್ಕೇ ಮ್ಯಾಟ್ರಿಕ್ಸ್ ಉಪಯುಕ್ತ. (MNIST ನಲ್ಲಿ ಇದು 10×10 ಮ್ಯಾಟ್ರಿಕ್ಸ್ ಆಗುತ್ತದೆ; ಉದಾ: "4" ಮತ್ತು "9" ಹೆಚ್ಚು ಗೊಂದಲ ಉಂಟುಮಾಡುತ್ತವೆ.)

೯. Train / Validation / Test ಮತ್ತು Hyperparameters

ಮಾದರಿಯನ್ನು **ತರಬೇತಿ ಮಾಡಿದ ದತ್ತಾಂಶದಲ್ಲೇ** ಪರೀಕ್ಷಿಸಿದರೆ ಅದು ನಿಜವಾದ ಸಾಮರ್ಥ್ಯ ತೋರಿಸುವುದಿಲ್ಲ — ಉರು ಹೊಡೆದಿರಬಹುದು. ಹಾಗಾಗಿ ದತ್ತಾಂಶವನ್ನು ವಿಭಜಿಸುತ್ತೇವೆ:

ಭಾಗ	ಉಪಯೋಗ (Goodfellow, Bengio, Courville 2016)	ಶಾಲೆಯ ಹೋಲಿಕೆ
Training set	ಮಾದರಿಯ ತೂಕಗಳನ್ನು (parameters) ಕಲಿಸಲು	ಪಠ್ಯಪುಸ್ತಕದ ಅಭ್ಯಾಸ ಪ್ರಶ್ನೆಗಳು
Validation set	Hyperparameters ಟ್ಯೂನ್ ಮಾಡಲು, ಉತ್ತಮ ಮಾದರಿ ಆರಿಸಲು, ತರಬೇತಿ ಸಮಯದ generalization error ಅಂದಾಜಿಸಲು	ಮಾದರಿ ಪ್ರಶ್ನೆಪತ್ರಿಕೆ (ಪ್ರಿ-ಪರೀಕ್ಷೆ) — ತಪ್ಪು ನೋಡಿ ತಯಾರಿ ಬದಲಿಸಿಕೊಳ್ಳುತ್ತೇವೆ
Test set	ಸಂಪೂರ್ಣವಾಗಿ ಸಿದ್ಧವಾದ ಮಾದರಿಯ ಅಂತಿಮ ಮೌಲ್ಯಮಾಪನಕ್ಕೆ ಮಾತ್ರ	ಅಂತಿಮ ಪರೀಕ್ಷೆ — ಒಮ್ಮೆ ಮಾತ್ರ ಬರೆಯುತ್ತೇವೆ

> **ಚಿನ್ನದ ನಿಯಮ**
> Test set ಅನ್ನು ಮಾದರಿ ಟ್ಯೂನ್ ಮಾಡಲು ಬಳಸಬೇಡಿ. ಪರೀಕ್ಷೆಯ ಪ್ರಶ್ನೆಪತ್ರಿಕೆಯನ್ನು ಮುಂಚೆಯೇ ನೋಡಿ ತಯಾರಾದರೆ ಅಂಕ ನಿಜವಾದ ಸಾಮರ್ಥ್ಯ ತೋರಿಸುವುದಿಲ್ಲ — ಹಾಗೆಯೇ.

Parameters (ಪ್ಯಾರಾಮೀಟರ್‌ಗಳು)	Hyperparameters (ಹೈಪರ್‌ಪ್ಯಾರಾಮೀಟರ್‌ಗಳು)
ನೆಟ್‌ವರ್ಕ್ **ತಾನೇ ಕಲಿಯುವ** ಸಂಖ್ಯೆಗಳು — ಸಂಪರ್ಕ ತೂಕಗಳು (weights) ಮತ್ತು bias	ನಾವು **ಕೈಯಾರೆ ನಿಗದಿಪಡಿಸುವ** ಆಯ್ಕೆಗಳು: ಪದರಗಳ ಸಂಖ್ಯೆ, epochs ಸಂಖ್ಯೆ, mini-batch size, learning rate, ನೆಟ್‌ವರ್ಕ್ ವಿನ್ಯಾಸ

> **ಹೋಲಿಕೆ**
> ವಿದ್ಯಾರ್ಥಿ ಕಲಿತ ಜ್ಞಾನ = parameters. ಪ್ರತಿದಿನ ಎಷ್ಟು ಗಂಟೆ ಓದಬೇಕು, ಯಾವ ಪುಸ್ತಕ ಆರಿಸಬೇಕು ಎಂಬ ನಿರ್ಧಾರ = hyperparameters (ಇವನ್ನು ವಿದ್ಯಾರ್ಥಿ/ಶಿಕ್ಷಕ ನಿರ್ಧರಿಸುತ್ತಾರೆ, ಜ್ಞಾನ ಅಲ್ಲ).

೧೦. Learning Curves, Overfitting ಮತ್ತು Early Stopping

Overfitting ಮತ್ತು Underfitting

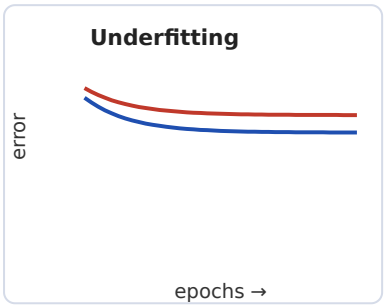
ಸಮಸ್ಯೆ	ಕಾರಣ	ಲಕ್ಷಣ
Underfitting	ಮಾದರಿ ತುಂಬ ಸರಳ	ತರಬೇತಿ ದತ್ತಾಂಶದಲ್ಲೇ ಕೆಟ್ಟ ಫಲಿತಾಂಶ; ಮುಖ್ಯ ಮಾದರಿಗಳನ್ನು ಹಿಡಿಯುವುದಿಲ್ಲ
Overfitting	ಮಾದರಿ ತುಂಬ ಸಂಕೀರ್ಣ (ದತ್ತಾಂಶಕ್ಕೆ ಹೋಲಿಸಿದರೆ ತುಂಬ ಪ್ಯಾರಾಮೀಟರ್‌ಗಳು)	ತರಬೇತಿಯಲ್ಲಿ ಅತ್ಯುತ್ತಮ (noise ಸಹಿತ ಉರು), ಹೊಸ ದತ್ತಾಂಶದಲ್ಲಿ ಕೆಟ್ಟ

ನೆನಪಿಡಿ: ಕೇವಲ ತರಬೇತಿ ದತ್ತಾಂಶವನ್ನು ಉರು ಹೊಡೆಯುವ ನೆಟ್‌ವರ್ಕ್ ಹೊಸದನ್ನು ಸರಿಯಾಗಿ ವರ್ಗೀಕರಿಸಲಾರದು. ನ್ಯೂರಾನ್/ಪ್ಯಾರಾಮೀಟರ್ ಸಂಖ್ಯೆ ಕಾರ್ಯದ ಸಂಕೀರ್ಣತೆಗೆ ತಕ್ಕಂತಿರಬೇಕು — **ತುಂಬ ಹೆಚ್ಚೂ ಬೇಡ, ತುಂಬ**

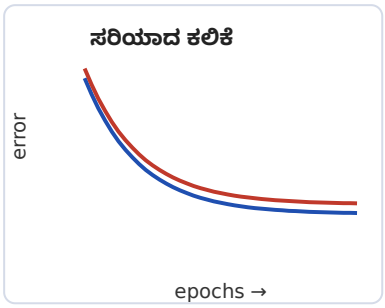
ಕಡಿಮೆಯೂ ಬೇಡ.

Learning curves — ಕಲಿಕೆಯ ನಾಡಿಮಿಡಿತ

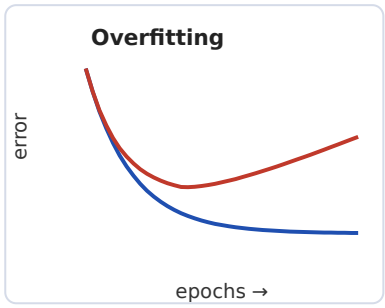
X ಅಕ್ಷದಲ್ಲಿ epochs, Y ಅಕ್ಷದಲ್ಲಿ error (loss). ನೀಲಿ = training, ಕೆಂಪು = validation. ಈ ಎರಡು ರೇಖೆಗಳ ಆಕಾರವೇ ರೋಗನಿರ್ಣಯ:



ಎರಡೂ ರೇಖೆಗಳು ಎತ್ತರದಲ್ಲಿ



ಎರಡೂ ಕೆಳಗೆ, ಹತ್ತಿರ ಹತ್ತಿರ



Training ಕೆಳಗೆ, Validation ಮತ್ತೆ ಮೇಲೆ

ಚಿತ್ರ	ಏನು ಹೇಳುತ್ತದೆ	ಪರಿಹಾರ
Underfitting	ತರಬೇತಿ error ಕೂಡ ಹೆಚ್ಚು	ದೊಡ್ಡ ಮಾದರಿ, ಹೆಚ್ಚು epochs, ಉತ್ತಮ features
ಸರಿಯಾದ ಕಲಿಕೆ	ಎರಡೂ ಕಡಿಮೆ, ಅಂತರ ಚಿಕ್ಕದು	ಹಾಗೆಯೇ ಮುಂದುವರಿಸಿ
Overfitting	Validation error ಒಂದು ಹಂತದ ನಂತರ ಮತ್ತೆ ಏರುತ್ತದೆ	Early stopping, ಚಿಕ್ಕ ಮಾದರಿ, ಹೆಚ್ಚು ದತ್ತಾಂಶ

Early Stopping — “ಸರಿಯಾದ ಸಮಯದಲ್ಲಿ ನಿಲ್ಲಿಸು”

Validation error ಏರಲು ಆರಂಭಿಸಿದ ಕ್ಷಣದಲ್ಲಿ ತರಬೇತಿ ನಿಲ್ಲಿಸಿ, ಆ ಕ್ಷಣದ “ಅತ್ಯುತ್ತಮ ಮಾದರಿ”ಯನ್ನು ಉಳಿಸಿಕೊಳ್ಳುವುದು. ಜೆಫ್ರಿ ಹಿಂಟನ್ ಇದನ್ನು ಹಾಸ್ಯವಾಗಿ “ಉಚಿತ ತಿಂಡಿ” (a free breakfast/lunch) ಎಂದು ಕರೆದರು — ಏಕೆಂದರೆ ಇದಕ್ಕೆ ಹೆಚ್ಚುವರಿ ವೆಚ್ಚವಿಲ್ಲ, ಲಾಭ ಮಾತ್ರ.

> **ಆದರೆ ಎಚ್ಚರ**
>
> Early stopping ಬಳಸಲು ಸಹ ಮಾದರಿಯನ್ನು **ಸಾಕಷ್ಟು epochs** ತರಬೇತಿ ಮಾಡಬೇಕು. ಕೇವಲ 2–3 epochs ನಂತರ ನಿಲ್ಲಿಸಿದರೆ ಮಾದರಿ ಏನನ್ನೂ ಕಲಿತಿರುವುದಿಲ್ಲ (ಅದು underfitting). ಅಂದರೆ: ಮೊದಲು ಸಾಕಷ್ಟು ಕಲಿಯಲು ಬಿಡಿ, validation ಕೆಟ್ಟಾಗ ನಿಲ್ಲಿಸಿ.

೧೧. Keras: Sequential, Input, Flatten, Dense

Keras ನ್ಯೂರಲ್ ನೆಟ್‌ವರ್ಕ್‌ಗಳನ್ನು ಸುಲಭವಾಗಿ ಕಟ್ಟಲು ಬಳಸುವ Python ಲೈಬ್ರರಿ (ಈ ಉಪನ್ಯಾಸದಲ್ಲಿ Keras 3). ನೆಟ್‌ವರ್ಕ್ ಅನ್ನು ಲೆಗೋ ಬ್ಲಾಕ್‌ಗಳಂತೆ *ಪದರ-ಪದರವಾಗಿ* ಜೋಡಿಸುತ್ತೇವೆ. ನಮ್ಮ ಗುರಿ: 28×28 ಚಿತ್ರದಲ್ಲಿ ಯಾವ

ಅಂಕೆ ಇದೆ ಎಂದು ಊಹಿಸುವುದು (784 ಇನ್‌ಪುಟ್ → 10 ಔಟ್‌ಪುಟ್).

ಎರಡು ವಿಧಾನಗಳು

API	ಯಾವಾಗ
Sequential API	ಸರಳ, ರೇಖೀಯ ಸ್ಟ್ಯಾಕ್ — ಪ್ರತಿ ಪದರ ನೇರವಾಗಿ ಮುಂದಿನ ಪದರಕ್ಕೆ ನೀಡುತ್ತದೆ (Dense, Conv1D/2D, LSTM, GRU ಮುಂತಾದ ಪದರಗಳಿಗೆ ಸರಿ). ಇಲ್ಲಿ ಇದನ್ನೇ ಬಳಸುತ್ತೇವೆ.
Functional API	ಹೆಚ್ಚು ಸಂಕೀರ್ಣ ವಿನ್ಯಾಸ (ಹಲವು ಇನ್‌ಪುಟ್/ಔಟ್‌ಪುಟ್, ಕವಲು, ಜೋಡಣೆ).

```
# Sequential ವಿಧಾನ: ಪದರಗಳ ಪಟ್ಟಿ ಕೊಡಿ
from keras.models import Sequential
from keras.layers import Dense, Input
model = Sequential([
    Input(shape=(784,)),
    Dense(128, activation='relu'),
    Dense(10, activation='softmax')
])

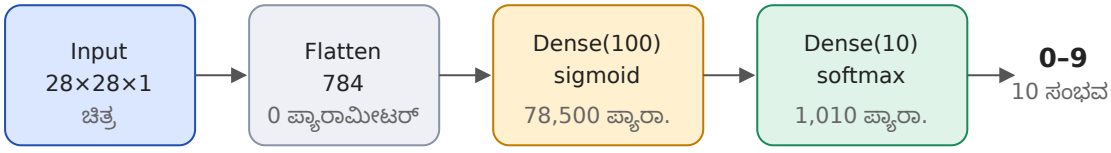
# Functional ವಿಧಾನ: ಪದರಗಳನ್ನು ಕರೆದಂತೆ ಒಂದಕ್ಕೊಂದು ಜೋಡಿಸಿ
from keras.models import Model
inputs  = Input(shape=(784,))
x       = Dense(128, activation='relu')(inputs)
outputs = Dense(10, activation='softmax')(x)
model   = Model(inputs, outputs)
```

ಮೂರು ಮುಖ್ಯ ಪದರಗಳು

ಪದರ	ಕೆಲಸ	ಟಿಪ್ಪಣಿ
Input	ಒಂದು ಉದಾಹರಣೆಯ **ಆಕಾರ (shape)** ಮಾತ್ರ ಹೇಳುತ್ತದೆ; ತೂಕ ಇಲ್ಲ, ದತ್ತಾಂಶ ಬದಲಿಸುವುದಿಲ್ಲ	Keras 3 ನಲ್ಲಿ `Input(...)` ಸ್ಪಷ್ಟವಾಗಿ ಬರೆಯಬೇಕು. ಉದಾ: (28,28,1) = ಗ್ರೇಸ್ಕೇಲ್ ಚಿತ್ರ; (32,32,3) = RGB ಚಿತ್ರ; (100,) = ಟೇಬಲ್ ದತ್ತಾಂಶ
Flatten	ಬಹು-ಆಯಾಮದ ದತ್ತಾಂಶವನ್ನು (ಉದಾ: 28×28) ಒಂದು ಸಾಲಿಗೆ (784) ಚಪ್ಪಟೆ ಮಾಡುತ್ತದೆ	ಪ್ರತಿ ಉದಾಹರಣೆಯ ಆಕಾರ ಮಾತ್ರ ಬದಲಾಗುತ್ತದೆ, batch size ಅಲ್ಲ. 0 ಪ್ಯಾರಾಮೀಟರ್
Dense (fully connected)	ಪ್ರತಿ ನ್ಯೂರಾನ್ ಹಿಂದಿನ ಪದರದ **ಎಲ್ಲ** ನ್ಯೂರಾನ್‌ಗಳೊಂದಿಗೆ ಸಂಪರ್ಕ	ಮರೆ ಮತ್ತು ಔಟ್‌ಪುಟ್ ಪದರಗಳಲ್ಲಿ ಬಳಕೆ. Input (batch_size, input_dim) → Output (batch_size, units); ಉದಾ: (32, 1000) → (32, 10)

> **Batch ಆಯಾಮದ ಬಗ್ಗೆ**
> ತರಬೇತಿಯಲ್ಲಿ ಒಂದು ಚಿತ್ರವನ್ನಲ್ಲ, 32 ಅಥವಾ 64 ಚಿತ್ರಗಳ **ಗುಂಪನ್ನು** (batch) ಒಮ್ಮೆಗೆ ಕಳುಹಿಸುತ್ತೇವೆ. ಆದ್ದರಿಂದ ಆಂತರಿಕ ಪೂರ್ಣ ಆಕಾರ (batch_size, height, width, channels). ಆದರೆ `Input(shape=(28, 28, 1))` ನಲ್ಲಿ ನಾವು **ಒಂದೇ ಉದಾಹರಣೆಯ** ಆಕಾರ ಮಾತ್ರ ಬರೆಯುತ್ತೇವೆ — batch size ಅನ್ನು Keras ತಾನೇ ಸೇರಿಸುತ್ತದೆ.

ನಮ್ಮ MNIST ನೆಟ್‌ವರ್ಕ್



ಒಟ್ಟು ಪ್ಯಾರಾಮೀಟರ್‌ಗಳು = 0 + 78,500 + 1,010 = 79,510

ಚಿತ್ರ ೩: 784 ಇನ್‌ಪುಟ್ → 100 ನ್ಯೂರಾನ್ (ಮರೆ ಪದರ) → 10 ಔಟ್‌ಪುಟ್

ಪದರಗಳ ಹೆಸರಿನ ಕೆಳಗಿನ "ಪ್ಯಾರಾಮೀಟರ್" ಸಂಖ್ಯೆಗಳು ಹೇಗೆ ಬಂದವು ಎಂದು ಭಾಗ ೧೩ ರಲ್ಲಿ ನೋಡುತ್ತೇವೆ.

೧೨. ಮಾದರಿ ನಿರ್ಮಾಣದ ಜೀವನಚಕ್ರ ಮತ್ತು MNIST ಸಮಸ್ಯೆ

ಎಂಟು ಹಂತಗಳು

ಹಂತ	ಕೆಲಸ	Keras ಕೋಡ್
೧	ಲೈಬ್ರರಿ import	`from keras import layers`
೨	ದತ್ತಾಂಶ ಲೋಡ್	`mnist.load_data()`
೩	ಮಾದರಿ ನಿರ್ಮಿಸು	`model = Sequential([...])`
೪	Compile (loss, optimizer, metrics)	`model.compile(...)`
೫	ತರಬೇತಿ	`model.fit(...)`
೬	ಕಲಿಕೆಯ ಪ್ರಗತಿ ಗಮನಿಸು (ಬೇಕಾದರೆ ೩–೫ ರ hyperparameters ಬದಲಿಸು)	`plt.plot(...)`
೭	Test ದತ್ತಾಂಶದಲ್ಲಿ ಮೌಲ್ಯಮಾಪನ	`model.evaluate(...)`
೮	ಹೊಸ ದತ್ತಾಂಶಕ್ಕೆ ಭವಿಷ್ಯವಾಣಿ	`model.predict(...)`

MNIST — ಕಂಪ್ಯೂಟರ್‌ಗೆ ಅಂಕೆ ಓದಲು ಕಲಿಸುವುದು

MNIST (Modified National Institute of Standards and Technology) ಕೈಬರಹದ ಅಂಕೆಗಳ (0–9) ಸಾರ್ವಜನಿಕ ದತ್ತಾಂಶ: **60,000 ತರಬೇತಿ** ಚಿತ್ರಗಳು (≈ 86%) ಮತ್ತು **10,000 test** ಚಿತ್ರಗಳು (≈ 14%). ಪ್ರತಿ ಚಿತ್ರ 28×28 ಪಿಕ್ಸೆಲ್ = **784 ಇನ್‌ಪುಟ್ ಲಕ್ಷಣಗಳು**. ಕೆಲಸ: ವರ್ಗೀಕರಣ — ಯಾವ ಅಂಕೆ ಇದೆ? ಒಳ್ಳೆಯ ತರಬೇತಿ ದತ್ತಾಂಶ ದೊಡ್ಡದು, ವೈವಿಧ್ಯಮಯ ಮತ್ತು ಸಮತೋಲಿತವಾಗಿರಬೇಕು.

> **ಕಥೆ: ಮಗುವಿಗೆ ಅಂಕೆ ಕಲಿಸುವುದು**
>
> ಪುಟ್ಟ ಮಗುವಿಗೆ '7' ಹೇಗಿರುತ್ತದೆ ಎಂದು ಕಲಿಸಲು ನೂರಾರು ಬೇರೆ ಬೇರೆ ಕೈಬರಹದ '7' ತೋರಿಸುತ್ತೇವೆ — ಕೆಲವು ಓರೆ, ಕೆಲವು ದಪ್ಪ, ಕೆಲವು ಗೆರೆ ಹಾಕಿದ್ದು. ಒಂದೇ ರೀತಿಯ '7' ಮಾತ್ರ ತೋರಿಸಿದರೆ ಬೇರೆ ಕೈಬರಹ ಗುರುತಿಸಲಾರದು. ಅದಕ್ಕೇ ದತ್ತಾಂಶ ವೈವಿಧ್ಯಮಯವಾಗಿರಬೇಕು.

ಚಿತ್ರ → ಸಂಖ್ಯೆಗಳು

- ಪ್ರತಿ ಪಿಕ್ಸೆಲ್ 0 ರಿಂದ 255 ರ ಸಂಖ್ಯೆ: **0 = ಕಪ್ಪು, 255 = ಬಿಳಿ** (255 = 11111111 ಬೈನರಿ, ಅಂದರೆ 8 ಬಿಟ್).
- **Normalisation:** ಎಲ್ಲ ಪಿಕ್ಸೆಲ್‌ಗಳನ್ನು 255 ರಿಂದ ಭಾಗಿಸಿ 0–1 ವ್ಯಾಪ್ತಿಗೆ ತರುತ್ತೇವೆ (ಉದಾ: 128 → 0.502). ಇದರಿಂದ ಕಲಿಕೆ ಸ್ಥಿರ ಮತ್ತು ವೇಗ (ಹಿಂದಿನ ಉಪನ್ಯಾಸದ ಇನ್‌ಪುಟ್ normalization).

ಲೇಬಲ್ → One-hot ಮತ್ತು Softmax ಹೋಲಿಕೆ

ಔಟ್‌ಪುಟ್ ಪದರ (softmax) 10 ಸಂಭವನೀಯತೆಗಳ ಪಟ್ಟಿ ಕೊಡುತ್ತದೆ. ಆದರೆ ಸರಿಯಾದ ಉತ್ತರ ಒಂದೇ ಸಂಖ್ಯೆ (ಉದಾ: 7). ಈ ಎರಡನ್ನು ನೇರವಾಗಿ ಹೋಲಿಸಲಾಗುವುದಿಲ್ಲ ("ಗುಣ ಬೇರೆ"). ಅದಕ್ಕೆ ಲೇಬಲ್ ಅನ್ನು ಅದೇ ರೂಪಕ್ಕೆ ಬದಲಿಸುತ್ತೇವೆ — **one-hot vector**: ಸರಿಯಾದ ಸ್ಥಾನದಲ್ಲಿ 1, ಉಳಿದೆಡೆ 0.

ಲೇಬಲ್	One-hot (ಸ್ಥಾನ 0 ರಿಂದ 9)
1	[0, **1**, 0, 0, 0, 0, 0, 0, 0, 0]
7	[0, 0, 0, 0, 0, 0, 0, **1**, 0, 0]
2	[0, 0, **1**, 0, 0, 0, 0, 0, 0, 0]

> **Categorical cross-entropy ಲೆಕ್ಕ**
>
> ಸ್ಲೈಡ್‌ನ ಕೊನೆಯ ಭವಿಷ್ಯವಾಣಿಯಲ್ಲಿ ಮಾದರಿ '7' ಚಿತ್ರಕ್ಕೆ ಕೊಟ್ಟ ಸಂಭವನೀಯತೆಗಳು: [0, 0.133, 0.274, 0.006, 0, 0.002, 0.001, **0.573**, 0.009, 0.002]. ನಿಜ ಲೇಬಲ್ one-hot ಪ್ರಕಾರ '7' ಸ್ಥಾನದಲ್ಲಿ ಮಾತ್ರ 1:
>
> ```
> loss = −ln( ಸರಿಯಾದ ವರ್ಗಕ್ಕೆ ಕೊಟ್ಟ ಸಂಭವನೀಯತೆ ) = −ln(0.573) = 0.557
> ```
>
> ಮಾದರಿ 100% ಖಚಿತವಾಗಿದ್ದರೆ (p = 1) loss = 0. ಸಂಭವನೀಯತೆ ಕಡಿಮೆಯಾದಷ್ಟೂ loss ಹೆಚ್ಚು. (ಹಿಂದಿನ ಉಪನ್ಯಾಸ ನೋಡಿ.) 60,000 ಚಿತ್ರಗಳ ತರಬೇತಿ ದತ್ತಾಂಶದಲ್ಲಿ ಪ್ರತಿ ಚಿತ್ರಕ್ಕೂ [784 ಪಿಕ್ಸೆಲ್ ಮೌಲ್ಯಗಳು] + [10 ಸಂಖ್ಯೆಯ one-hot ಟ್ಯಾಗ್] ಜೋಡಿ ಇರುತ್ತದೆ.

೧೩. ಕೋಡ್ ಹಂತ-ಹಂತವಾಗಿ: ಡೇಟಾದಿಂದ ಭವಿಷ್ಯವಾಣಿವರೆಗೆ

ಸ್ಲೈಡ್‌ಗಳ ಕೋಡ್ ಅನ್ನು ಹಂತ ಹಂತವಾಗಿ ಕನ್ನಡ ಟಿಪ್ಪಣಿಗಳೊಂದಿಗೆ ನೀಡಿದ್ದೇವೆ (ಟಿಪ್ಪಣಿಗಳು `#` ನಂತರ).

ಹಂತ ೧–೨: Import ಮತ್ತು ದತ್ತಾಂಶ ಲೋಡ್

```
import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt
import keras
from keras.layers import Dense, Flatten, Input

mnist = tf.keras.datasets.mnist
(x_train, y_train), (x_test, y_test) = mnist.load_data()
# x_train: 60,000 ಚಿತ್ರಗಳು (60000, 28, 28)  | y_train: ಅವುಗಳ ನಿಜವಾದ ಅಂಕೆ (0–9)
# x_test : 10,000 ಚಿತ್ರಗಳು (10000, 28, 28)  | y_test : ಅವುಗಳ ನಿಜವಾದ ಅಂಕೆ
```

ಹಂತ ೩: ಕೆಲವು ಚಿತ್ರಗಳನ್ನು ನೋಡಿ

```
plt.figure(figsize=(10, 5))        # ಇಡೀ ಚಿತ್ರದ ಗಾತ್ರ
for i in range(15):                # ಮೊದಲ 15 ಚಿತ್ರಗಳು
    plt.subplot(3, 5, i + 1)       # 3 ಸಾಲು × 5 ಕಾಲಮ್ ಗ್ರಿಡ್
    plt.xticks([]); plt.yticks([]) # ಅಕ್ಷದ ಸಂಖ್ಯೆಗಳನ್ನು ತೆಗೆಯಿರಿ
    plt.imshow(x_train[i], cmap=plt.cm.binary)   # ಕಪ್ಪು-ಬಿಳಿ ಚಿತ್ರ
plt.show()
```

ಹಂತ ೪: Normalisation ಮತ್ತು One-hot

```
x_train = x_train / 255     # ಪಿಕ್ಸೆಲ್ 0–255 → 0–1
x_test  = x_test  / 255

from keras.utils import to_categorical
y_train2 = to_categorical(y_train, 10)   # 7 → [0,0,0,0,0,0,0,1,0,0]
y_test2  = to_categorical(y_test, 10)
```

ಹಂತ ೫: ಮಾದರಿ ನಿರ್ಮಾಣ

```
model = keras.Sequential([
    Input(shape=(28, 28, 1)),              # ಒಂದು ಚಿತ್ರದ ಆಕಾರ: 28×28, 1 ಚಾನೆಲ್ (ಗ್ರೇಸ್ಕೇಲ್)
    Flatten(),                             # 28×28 → 784 ಸಂಖ್ಯೆಗಳ ಸಾಲು
    Dense(100, activation='sigmoid'),      # ಮರೆ ಪದರ: 100 ನ್ಯೂರಾನ್
    Dense(10, activation='softmax')        # ಔಟ್‌ಪುಟ್: 10 ಅಂಕೆಗಳ ಸಂಭವನೀಯತೆ
])
print(model.summary())                     # ಪದರಗಳು ಮತ್ತು ಪ್ಯಾರಾಮೀಟರ್ ಸಂಖ್ಯೆ ತೋರಿಸುತ್ತದೆ
```

ಮಾದರಿಯಲ್ಲಿ ಎಷ್ಟು ಪ್ಯಾರಾಮೀಟರ್‌ಗಳಿವೆ? (ಸ್ಲೈಡ್‌ನ ಲೆಕ್ಕ)

```
Flatten      : 0                                          (ಆಕಾರ ಬದಲಾವಣೆ ಮಾತ್ರ)
Dense(100)   : 28×28 = 784 ಇನ್‌ಪುಟ್; +1 (bias) = 785
               785 × 100 = 78,500   (= 784×100 ತೂಕ + 100 bias)
Dense(10)    : 100 ಇನ್‌ಪುಟ್; +1 (bias) = 101
               101 × 10  = 1,010    (= 100×10 ತೂಕ + 10 bias)
ಒಟ್ಟು        : 78,500 + 1,010 = 79,510  (ಎಲ್ಲವೂ trainable, non-trainable = 0)
```

ಕೇವಲ ಸಣ್ಣ ನೆಟ್‌ವರ್ಕ್‌ಗೂ ಸುಮಾರು 80 ಸಾವಿರ ತೂಕಗಳನ್ನು ಕಲಿಯಬೇಕು!

ಹಂತ ೬: Compile — loss ಮತ್ತು optimizer

```
opt = keras.optimizers.Adam(learning_rate=0.001)   # ಆಪ್ಟಿಮೈಜರ್: 'sgd', 'rmsprop', 'adam' ಆಯ್ಕೆಗಳು
model.compile(optimizer=opt,
              loss='categorical_crossentropy',      # one-hot ಲೇಬಲ್‌ಗೆ ಸರಿಯಾದ loss
              metrics=['accuracy'])                 # ತರಬೇತಿ ವೇಳೆ ತೋರಿಸಬೇಕಾದ ಅಳತೆ
```

ಹಂತ ೭: ತರಬೇತಿ — fit

```
training = model.fit(x_train, y_train2,
                     batch_size=32,         # 32 ಚಿತ್ರಗಳ ಪ್ರತಿ mini-batch ನಂತರ ತೂಕ ಬದಲಾವಣೆ
                     epochs=6,              # ಇಡೀ ದತ್ತಾಂಶದ 6 ಪೂರ್ಣ ಸುತ್ತುಗಳು
                     validation_split=0.2)  # 20% ದತ್ತಾಂಶವನ್ನು validation ಗೆ ಮೀಸಲು
```

ಒಂದು epoch ನಲ್ಲಿ ಎಷ್ಟು ಹೆಜ್ಜೆ? (ಸ್ಲೈಡ್‌ನ ಲೆಕ್ಕ)

```
Validation : 60,000 × 0.2 = 12,000
Training   : 60,000 − 12,000 = 48,000
ಹೆಜ್ಜೆಗಳು/epoch : 48,000 / 32 = 1,500     (ಲಾಗ್‌ನಲ್ಲಿ "1500/1500")
6 epochs ನಲ್ಲಿ ಒಟ್ಟು ತೂಕ ಬದಲಾವಣೆ = 6 × 1,500 = 9,000
Test       : 10,000 / 32 = 312.5 → 313 ಹೆಜ್ಜೆ (ಲಾಗ್‌ನಲ್ಲಿ "313/313")
```

ಹಂತ ೮: ಮೌಲ್ಯಮಾಪನ ಮತ್ತು ಕಲಿಕೆಯ ವಕ್ರರೇಖೆಗಳು

```
test = model.evaluate(x_test, y_test2)      # test ದತ್ತಾಂಶದಲ್ಲಿ loss ಮತ್ತು accuracy

print(training.history.keys())              #
'loss','accuracy','val_loss','val_accuracy'
plt.plot(training.history['accuracy'])
plt.plot(training.history['val_accuracy'])
plt.legend(['training', 'validation'])
plt.show()

plt.plot(training.history['loss'])
plt.plot(training.history['val_loss'])      # ಸ್ಪ್ಲಿಡ್‌ನ ಅಭ್ಯಾಸ: validation loss ಕೂಡ ಸೇರಿಸಿ
plt.title('Model loss function')
plt.xlabel('epoch'); plt.ylabel('loss')
plt.legend(['training', 'validation'], loc='upper left')
plt.show()
```

ಹಂತ ೯: ಹೊಸ PNG ಚಿತ್ರದ ಅಂಕೆ ಗುರುತಿಸುವುದು

```
from PIL import Image
from numpy import asarray

image = Image.open("Sample7b.png")   # ಚಿತ್ರ ಲೋಡ್
img = image.convert('L')             # 'L' = ಗ್ರೇಸ್ಕೇಲ್ (RGBA ನ 4 ಚಾನೆಲ್ → 1 ಚಾನೆಲ್)
numpydata = asarray(img)             # PIL ಚಿತ್ರ → NumPy ಅರೇ (28, 28)
numpydata = numpydata / 255          # ತರಬೇತಿಯಂತೆಯೇ normalise ಮಾಡಿ

x = np.expand_dims(numpydata, axis=0)  # (28,28) → (1,28,28): ಒಂದು ಚಿತ್ರದ batch
spr = model.predict(x)                 # 10 ಸಂಭವನೀಯತೆಗಳು
print(np.round(spr, 3))
print(f"Number: {np.argmax(spr)}")     # ಅತಿ ಹೆಚ್ಚು ಸಂಭವನೀಯತೆಯ ಸ್ಥಾನ = ಊಹಿಸಿದ ಅಂಕೆ
plt.imshow(numpydata, cmap=plt.cm.binary); plt.show()
```

ಗಮನಿಸಿ

• PNG ಚಿತ್ರಗಳಲ್ಲಿ RGBA (Red, Green, Blue, Alpha = ಪಾರದರ್ಶಕತೆ 0.0–1.0) ಇರುತ್ತದೆ — ಹಾಗಾಗಿ ಆಕಾರ (28, 28, 4) ಬರುತ್ತದೆ; `convert('L')` ಇದನ್ನು ಗ್ರೇಸ್ಕೇಲ್ ಮಾಡುತ್ತದೆ.

• ಸ್ಪ್ಲಿಡ್‌ನ ಫಲಿತಾಂಶದಲ್ಲಿ '7' ಚಿತ್ರಕ್ಕೆ ಮಾದರಿ ಅತಿ ಹೆಚ್ಚು ಸಂಭವನೀಯತೆ (0.573) '7' ಗೇ ನೀಡಿತು ✓, ಆದರೆ '2' ಗೂ 0.274 ನೀಡಿತು — ಅಂದರೆ ಖಚಿತತೆ ಕೇವಲ 57%.

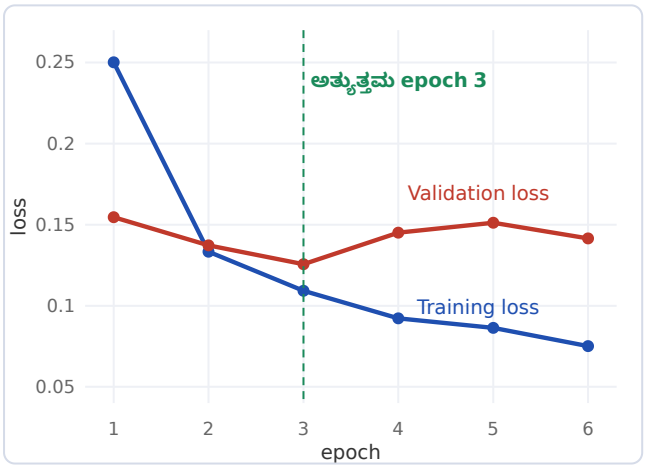
• (ನನ್ನ ಟಿಪ್ಪಣಿ) MNIST ಚಿತ್ರಗಳು ಕಪ್ಪು ಹಿನ್ನೆಲೆಯಲ್ಲಿ ಬಿಳಿ ಅಂಕೆ. ನಿಮ್ಮ ಸ್ವಂತ ಚಿತ್ರ ಬಿಳಿ ಹಿನ್ನೆಲೆಯಲ್ಲಿ ಕಪ್ಪು ಅಂಕೆ ಆಗಿದ್ದರೆ ಬಣ್ಣ ತಿರುವಿ (1 − ಮೌಲ್ಯ) ಬಳಸಿ ನೋಡಿ; ಮತ್ತು ಕೆಲವು Keras ಆವೃತ್ತಿಗಳಲ್ಲಿ ಆಕಾರ (1, 28, 28, 1) ಬೇಕಾಗಬಹುದು.

೧೪. ತರಬೇತಿ ಫಲಿತಾಂಶಗಳ ವಿಶ್ಲೇಷಣೆ (ಸ್ಲೈಡ್‌ನ ನಿಜವಾದ ಸಂಖ್ಯೆಗಳು)

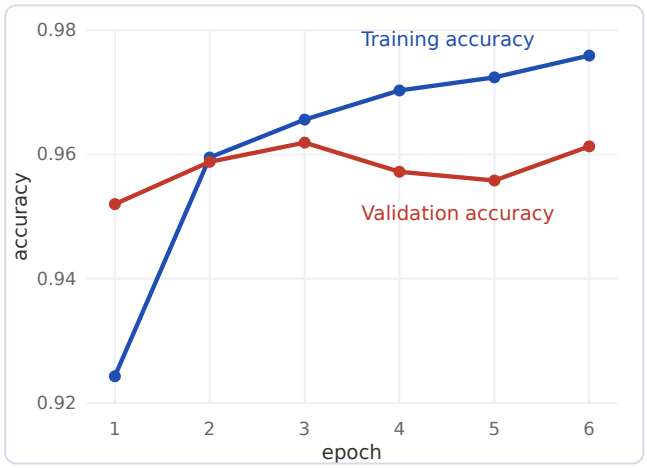
ಸ್ಲೈಡ್ 41 ರಲ್ಲಿ ತೋರಿಸಿದ ತರಬೇತಿ ಲಾಗ್ ಇದು (6 epochs, ಪ್ರತಿಯೊಂದರಲ್ಲಿ 1500 ಹೆಜ್ಜೆ):

Epoch	loss (train)	accuracy (train)	val_loss	val_accuracy
1	0.2502	0.9243	0.1546	0.9520
2	0.1334	0.9595	0.1373	0.9588
3	0.1093	0.9656	**0.1256** (ಕನಿಷ್ಠ)	**0.9619** (ಗರಿಷ್ಠ)
4	0.0923	0.9703	0.1451	0.9572
5	0.0864	0.9724	0.1512	0.9558
6	0.0751	0.9759	0.1416	0.9613

Test ಮೌಲ್ಯಮಾಪನ: loss = 0.1277, accuracy = 0.9629 — ಅಂದರೆ 10,000 test ಚಿತ್ರಗಳಲ್ಲಿ ಸುಮಾರು **9,629** ಸರಿಯಾಗಿ ಗುರುತಿಸಲ್ಪಟ್ಟವು (96.3%).



ಚಿತ್ರ ೪: Loss — training ಇಳಿಯುತ್ತಲೇ ಇದೆ, validation epoch 3 ರ ನಂತರ ಏರಿತು



ಚಿತ್ರ ೫: Accuracy — training ಏರುತ್ತಲೇ ಇದೆ, validation ಸ್ಥಗಿತ

ನಾವು ಏನು ಗಮನಿಸಬಹುದು?

• **Training loss ಪ್ರತಿ epoch ನಲ್ಲೂ ಇಳಿಯುತ್ತಿದೆ** (0.2502 → 0.0751) ಮತ್ತು training accuracy ಏರುತ್ತಿದೆ (92.4% → 97.6%).

• ಆದರೆ **validation loss epoch 3 ರಲ್ಲಿ ಕನಿಷ್ಠ (0.1256)**, ನಂತರ ಏರಿತು (0.1451, 0.1512) — ಮಾದರಿ ತರಬೇತಿ ದತ್ತಾಂಶವನ್ನು ಉರು ಹೊಡೆಯಲು ಆರಂಭಿಸಿದೆ ಎಂಬ **ಮೊದಲ ಎಚ್ಚರಿಕೆ ಚಿಹ್ನೆ (ಸೌಮ್ಯ overfitting)**.

• ಅಂತರ: epoch 6 ರಲ್ಲಿ train accuracy 97.59% vs validation 96.13% — ಸುಮಾರು 1.5 ಅಂಶ ವ್ಯತ್ಯಾಸ.

• Test accuracy (96.29%) ≈ validation accuracy (96.13%) — ಅಂದರೆ validation ಸರಿಯಾದ ಅಂದಾಜು ಕೊಟ್ಟಿತು. ಇದೇ validation ನ ಉಪಯೋಗ.

Early stopping ಇಲ್ಲಿ ಹೇಗೆ ಕೆಲಸ ಮಾಡುತ್ತಿತ್ತು? (ನನ್ನ ಸೇರ್ಪಡೆ — ಸ್ಲೈಡ್‌ನಲ್ಲಿ ಕೋಡ್ ಇಲ್ಲ)

“Validation loss ಸತತ 2 epochs ಸುಧಾರಿಸದಿದ್ದರೆ ನಿಲ್ಲಿಸು” ಎಂಬ ನಿಯಮ (patience = 2): epoch 3 ರಲ್ಲಿ ಅತ್ಯುತ್ತಮ (0.1256) → epoch 4 (0.1451) ಸುಧಾರಣೆ ಇಲ್ಲ (೧) → epoch 5 (0.1512) ಸುಧಾರಣೆ ಇಲ್ಲ (೨) → **ನಿಲ್ಲಿಸಿ** ಮತ್ತು epoch 3 ರ ತೂಕಗಳಿಗೆ ಹಿಂತಿರುಗಿ.

```
from keras.callbacks import EarlyStopping
stop = EarlyStopping(monitor='val_loss',      # ಯಾವುದನ್ನು ಗಮನಿಸಬೇಕು
                     patience=2,              # ಎಷ್ಟು epochs ಕಾಯಬೇಕು
                     restore_best_weights=True)  # ಅತ್ಯುತ್ತಮ epoch ನ ತೂಕ ಮರಳಿಸು
model.fit(x_train, y_train2, batch_size=32, epochs=20,
          validation_split=0.2, callbacks=[stop])
```

ಹಿಂಟನ್ ಎಚ್ಚರಿಕೆ ನೆನಪಿರಲಿ: ಮೊದಲ 2–3 epochs ನಲ್ಲೇ ಇದನ್ನು ಅತಿಯಾಗಿ ಬಿಗಿಯಾಗಿ ಇಡಬೇಡಿ; ಮಾದರಿಗೆ ಕಲಿಯಲು ಸಾಕಷ್ಟು ಸಮಯ (epochs) ಬೇಕು.

೧೫. ಸಾರಾಂಶ, ಪದಕೋಶ ಮತ್ತು ಅಭ್ಯಾಸ ಪ್ರಶ್ನೆಗಳು

ಸೂತ್ರಗಳ ಒಂದು-ಪುಟ ಸಾರಾಂಶ

ಅಳತೆ	ಸೂತ್ರ	ನಮ್ಮ 1000-ಜನರ ಉದಾಹರಣೆ (TP=80, FN=20, FP=45, TN=855)
Accuracy	(TP + TN) / ಒಟ್ಟು	0.935
Precision	TP / (TP + FP)	0.64
Recall / TPR / Sensitivity	TP / (TP + FN)	0.80
FPR	FP / (FP + TN)	0.05
Specificity	1 − FPR = TN / (TN + FP)	0.95
F1	2·P·R / (P + R)	0.711

ಪದಕೋಶ

ಇಂಗ್ಲಿಷ್	ಕನ್ನಡದಲ್ಲಿ ಸರಳ ಅರ್ಥ
Confusion matrix	ಊಹೆ vs ನಿಜ ಹೋಲಿಸುವ ಕೋಷ್ಟಕ (TP, FP, FN, TN)
False Negative (FN)	ರೋಗಿಯನ್ನು ಆರೋಗ್ಯವಂತ ಎಂದದ್ದು — ತಪ್ಪಿಸಿಕೊಂಡದ್ದು
False Positive (FP)	ಆರೋಗ್ಯವಂತನನ್ನು ರೋಗಿ ಎಂದದ್ದು — ಸುಳ್ಳು ಎಚ್ಚರಿಕೆ
Precision	“ಹೌದು” ಎಂದವುಗಳಲ್ಲಿ ಸರಿಯಾದವು
Recall	ನಿಜ “ಹೌದು” ಗಳಲ್ಲಿ ಹಿಡಿದವು
F1 score	Precision ಮತ್ತು Recall ನ ಹಾರ್ಮೋನಿಕ್ ಸರಾಸರಿ
Threshold	“ಹೌದು” ಎಂದು ಹೇಳಲು ಬೇಕಾದ ಕನಿಷ್ಠ ಸ್ಕೋರ್
ROC / AUC	ಎಲ್ಲ ಮಿತಿಗಳಲ್ಲಿ TPR vs FPR ವಕ್ರರೇಖೆ / ಅದರ ಕೆಳಗಿನ ವಿಸ್ತೀರ್ಣ
Training / Validation / Test set	ಕಲಿಯಲು / ಟ್ಯೂನ್ ಮಾಡಲು / ಅಂತಿಮ ಪರೀಕ್ಷೆಗೆ ದತ್ತಾಂಶ
Learning curve	epochs ಜೊತೆ training/validation error ಹೇಗೆ ಬದಲಾಗುತ್ತದೆ ಎಂದು ತೋರಿಸುವ ವಕ್ರರೇಖೆ
Early stopping	Validation ಕೆಟ್ಟಾಗ ತರಬೇತಿ ನಿಲ್ಲಿಸುವುದು
Parameters / Hyperparameters	ಕಲಿಯುವ ಸಂಖ್ಯೆಗಳು / ನಾವು ಹೊಂದಿಸುವ ಆಯ್ಕೆಗಳು
Sequential / Dense / Flatten / Input	ರೇಖೀಯ ಮಾದರಿ / ಪೂರ್ಣ-ಸಂಪರ್ಕ ಪದರ / ಚಪ್ಪಟೆ ಮಾಡುವ ಪದರ / ಆಕಾರ ಘೋಷಣೆ
Batch size / Epoch	ಒಮ್ಮೆಗೆ ಕಳುಹಿಸುವ ಉದಾಹರಣೆಗಳು / ಇಡೀ ದತ್ತಾಂಶದ ಒಂದು ಸುತ್ತು
One-hot encoding	ಸರಿಯಾದ ವರ್ಗದಲ್ಲಿ 1, ಉಳಿದೆಲ್ಲ 0 ಇರುವ ಸದಿಶ
MNIST	ಕೈಬರಹದ ಅಂಕೆಗಳ ಪ್ರಸಿದ್ಧ ದತ್ತಾಂಶ (60,000 + 10,000)

ಅಭ್ಯಾಸ ಪ್ರಶ್ನೆಗಳು

1. 1000 ಜನರಲ್ಲಿ 100 ರೋಗಿಗಳು; ಪರೀಕ್ಷೆಯಲ್ಲಿ TP = 70, FP = 10, FN = 30, TN = 890. Accuracy, Precision, Recall, F1 ಲೆಕ್ಕ ಹಾಕಿ.
2. ಕ್ಯಾನ್ಸರ್ ಸ್ಕ್ರೀನಿಂಗ್‌ನಲ್ಲಿ ಯಾವ ಅಳತೆ ಹೆಚ್ಚು ಮುಖ್ಯ ಮತ್ತು ಏಕೆ?
3. Precision = 0.5, Recall = 1.0 ಆದರೆ F1 ಎಷ್ಟು?
4. ಮ್ಯಾಟ್ರಿಕ್ಸ್ [[9,1,0],[1,8,1],[0,2,8]] ಗೆ accuracy ಎಷ್ಟು?
5. Flatten ನಂತರ Dense(50) ಪದರದಲ್ಲಿ ಎಷ್ಟು ಪ್ಯಾರಾಮೀಟರ್‌ಗಳು (28×28 ಇನ್‌ಪುಟ್)?
6. 60,000 ಚಿತ್ರಗಳು, validation_split = 0.1, batch_size = 64 ಆದರೆ ಒಂದು epoch ನಲ್ಲಿ ಎಷ್ಟು ಹೆಜ್ಜೆ?

7. Training loss ಇಳಿಯುತ್ತಿದ್ದು validation loss ಏರುತ್ತಿದ್ದರೆ ಇದರ ಅರ್ಥವೇನು? ಒಂದು ಪರಿಹಾರ ಹೇಳಿ.
8. Hyperparameters ಟ್ಯೂನ್ ಮಾಡಲು test set ಏಕೆ ಬಳಸಬಾರದು?

ಉತ್ತರಗಳು: (೧) Accuracy = 960/1000 = **96%**; Precision = 70/80 = **87.5%**; Recall = 70/100 = **70%**; F1 = 2×0.875×0.7/1.575 = **0.778**. (೨) **Recall** — ರೋಗ ತಪ್ಪಿಸಿಕೊಳ್ಳುವುದು (FN) ಅತಿ ಅಪಾಯಕಾರಿ. (೩) 2×0.5×1/1.5 = **0.667**. (೪) ಕರ್ಣ 9+8+8 = 25, ಒಟ್ಟು 30 → 25/30 = **83.3%**. (೫) 784×50 + 50 = **39,250**. (೬) 54,000 / 64 = 843.75 → **844** ಹೆಜ್ಜೆ. (೭) **Overfitting** — ಮಾದರಿ ಉರು ಹೊಡೆಯುತ್ತಿದೆ; ಪರಿಹಾರ: early stopping / ಚಿಕ್ಕ ಮಾದರಿ / ಹೆಚ್ಚು ದತ್ತಾಂಶ. (೮) ಟ್ಯೂನ್ ಮಾಡಿದರೆ test set ಮಾದರಿಗೆ “ಸೋರಿ” ಹೋಗುತ್ತದೆ; ಅಂತಿಮ ಅಂಕ ನಿಜವಾದ ಸಾಮರ್ಥ್ಯವನ್ನು ತೋರಿಸುವುದಿಲ್ಲ — ಟ್ಯೂನಿಂಗ್‌ಗೆ validation, ಅಂತಿಮ ಪರೀಕ್ಷೆಗೆ ಮಾತ್ರ test.